P0012R1
revises N4533 and P0012R0
Jens Maurer <Jens.Maurer@gmx.net>
2015-10-22

P0012R1: Make exception specifications be part of the type system, version 5

Introduction

The exception specification of a function was never considered a part of its type. Instead, 15.4 [except.spec] paragraphs 5 and 6 only restrict assignments and initializations of pointers and references to (member) functions. This leads to odd holes in the type system that have resulted in a number of core issues.

Core issue 92 asks the fundamental question "Should exception-specifications be part of the type system?" and gives this example:

```
void (*p)() throw(int);
void (**pp)() throw() = &p;   // not currently an error
```

This issue was closed as NAD in February 2014, because "EWG determined that no action should be taken on this issue" (quote from the core issues list). Little technical argument seems to have been presented; closing the issue was rather seen as warding off work.

Paper N1664 "Toward Improved Optimization Opportunities in C++0x" by Walter Brown and Marc Paterno suggested a `nothrow` qualifier to enhance optimization opportunities when using non-throwing functions. Section 7.2 of that paper described the type effects of the `nothrow` qualifier essentially identical to those of the `noexcept` qualifier as proposed in the present paper.

Core issue 1946 "exception-specifications vs pointer dereference" highlighted that clarifications for determining potential exceptions of an expression brought forward for core issue 1351 caused this example to be ill-formed, which was arguably well-formed under the earlier imprecise wording:

```
void (*p)() throw(int);
void (&r)() throw(int) = *p;  // ill-formed
```

A similar example involving templates is impossible to fix without changes to the type system:

```
template<typename T> T& f(T* p);
void (*p)() throw(int);
void (&r)() throw(int) = f(p);  // ill-formed
```

Core issue 2010 "exception-specifications and conversion operators" adds these examples:

```
struct S { typedef void (*p)(); operator p(); };
void (*q)() noexcept = S();

void (*r)() noexcept = []() noexcept {};
```

The recent work on a specification for transactional memory support (see N4265 "Transactional Memory Support for C++: Wording (revision 3)") required integrating the `transaction_safe` specifier into the types of functions, among other things considering it for implicit conversions and overload resolution. This paper clones that approach to integrate the presence or absence of a non-throwing exception-specification (called `noexcept`) into the types of functions, too.

The predecessor paper N4518 was presented to the WG21 plenary in Lenexa, but straw polls were postponed to the following meeting to give all interested parties a chance to review the effects, in particular on existing code.

Changes compared to P0012R0

- Rename "pointer to function type" (non-quoted) to "function pointer type"
- Make function pointer conversion a qualification adjustment in overload resolution.

Changes compared to N4533

- add a reference to N1664 (thanks to Walter Brown)
- actually address reference initialization in 8.5.3 [dcl.init.ref] (thanks to Alisdair Meredith)

- adjust example in 4.12 [conv.fctptr] to avoid non-throwing *dynamic-exception-specification* (thanks to Alisdair Meredith)
- add a section "library impact" (thanks to Alisdair Meredith)

Changes compared to N4518

- Fix 14.8.2.3 [temp.deduct.conv] and consider pointer to member function, too (thanks to Stephan T. Lavavej)

Changes compared to N4320

- Review by the Evolution and Core Working Groups during the Lenexa meeting of WG21 (May 2015)

Library impact

The `result_of` template is unaffected, because it does not inspect the function or function object "Fn" in detail; similar for the INVOKE meta-operation (20.9.2 [func.require]).

The function call operator for `reference_wrapper` could be noexcept if "T" is a noexcept function (see wording below) and all conversions for arguments are noexcept. This is already implied by the existing wording for `mem_fn` (20.9.11 [func.memfn]).

There should be a second partial specialization for `packaged_task` (30.6.9 [futures.task]), namely `packaged_task<R(Args...) noexcept>` (not proposed in the present paper).

It is an open issue how to propagate "noexcept" through `std::function`.

Feature-test macro

The recommended feature-test macro is `__cpp_noexcept_function_type`.

Wording

Clause 4 [conv]

Change section 4 [conv] paragraph 1:

- ...
- **Zero or one function pointer conversion.**
- Zero or one qualification conversion.

No change in section 4.3 [conv.func] paragraph 1:

> An lvalue of function type T can be converted to a prvalue of type "pointer to T". The result is a pointer to the function. [Footnote: ...]

Add a new section after section 4.11 [conv.mem]:

> **4.12 [conv.fctptr] Function pointer conversions**
>
> **A prvalue of type "pointer to `noexcept` function" can be converted to a prvalue of type "pointer to function". The result is a pointer to the function. A prvalue of type "pointer to member of type `noexcept` function" can be converted to a prvalue of type "pointer to member of type function". The result points to the member function.**
>
> **[Example:**
>
> ```
> void (*p)() throw(int);
> void (**pp)() noexcept = &p;   // error: cannot convert to pointer to noexcept function
>
> struct S { typedef void (*p)(); operator p(); };
> void (*q)() noexcept = S();   // error: cannot convert to pointer to noexcept function
> ```
>
> **-- end example]**

Clause 5 [expr] Expressions

Change in 5 [expr] paragraph 13:

> [Note: ...] The *composite pointer type* of two operands p1 and p2 having types T1 and T2, respectively, where at least

one is a pointer or pointer to member type or `std::nullptr_t`, is:

- ...
- if T1 or T2 is "pointer to cv1 void" and the other type is "pointer to cv2 T", "pointer to cv12 void", where cv12 is the union of cv1 and cv2 ;
- **if T1 or T2 is "pointer to `noexcept` function" and the other type is "pointer to function", where the function types are otherwise the same, "pointer to function";**
- ...

Change in 5.1.2 [expr.prim.lambda] paragraph 6:

The closure type for a non-generic *lambda-expression* with no *lambda-capture* has a public non-virtual non-explicit const conversion function to pointer to function with C++ language linkage (7.5 [dcl.link]) having the same parameter and return types as the closure type's function call operator. **The conversion is to "pointer to `noexcept` function" if the function call operator has a non-throwing exception specification.**

Change in 5.2.2 [expr.call] paragraph 1:

The postfix expression shall have function type or ~~pointer to function~~ **function pointer** type. For a call to a non-member function or to a static member function, the postfix expression shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (4.3) is suppressed on the postfix expression), or it shall have ~~pointer to function~~ **function pointer** type.

Change in 8.3.1 [dcl.ptr] paragraph 4:

[Note: Forming a pointer to reference type is ill-formed; see 8.3.2. Forming a ~~pointer to function~~ **function pointer** type is ill-formed if the function type has cv-qualifiers or a ref-qualifier; see 8.3.5. Since the address of a bit-field (9.6) cannot be taken, a pointer can never point to a bit-field. â€" end note]

Change in 5.2.9 [expr.static.cast] paragraph 7:

The inverse of any standard conversion sequence (Clause 4 [conv]) not containing an lvalue-to-rvalue (4.1 [conv.lval]), array-to-pointer (4.2 [conv.array]), function-to-pointer (4.3), null pointer (4.10), null member pointer (4.11), ~~or~~ boolean (4.12)**, or function pointer (4.12 [conv.fctptr])** conversion, can be performed explicitly using `static_cast`. ...

Change in 5.10 [expr.eq] paragraph 2:

If at least one of the operands is a pointer, pointer conversions (4.10 [conv.ptr])**, function pointer conversions (4.12 [conv.fctptr]),** and qualification conversions (4.4 [conv.qual]) are performed on both operands to bring them to their composite pointer type (clause 5 [expr]).

Change in 5.16 [expr.cond] paragraph 6:

- One or both of the second and third operands have pointer type; pointer conversions (4.10 [conv.ptr])**, function pointer conversions (4.12 [conv.fctptr]),** and qualification conversions (4.4 [conv.qual]) are performed to bring them to their composite pointer type (5 [expr]). ...
- ...

Change in 5.17 [expr.throw]:

Evaluating a *throw-expression* with an operand throws an exception (15.1 [except.throw]); the type of the exception object is determined by removing any top-level cv-qualifiers from the static type of the operand and adjusting the type from "array of T" or ~~"function returning T"~~ **function type T** to "pointer to T"~~ or "pointer to "function returning T," respectively~~.

Change in 5.20 [expr.const] paragraph 3:

... A *converted constant expression* of type T is an expression, implicitly converted to a prvalue of type T, where the converted expression is a core constant expression and the implicit conversion sequence contains only user-defined conversions, lvalue-to-rvalue conversions (4.1 [conv.lval]), integral promotions (4.5 [conv.prom]), ~~and~~ integral conversions (4.7 [conv.integral]) other than narrowing conversions (8.5.4 [dcl.init.list])**, and function pointer conversions (4.12 [conv.fctptr])**. [Note: ...]

Section 8 [dcl.decl] Declarators

Change in 8.3.5 [dcl.fct] paragraphs 1 and 2:

In a declaration T D where D has the form

```
D1 ( parameter-declaration-clause ) cv-qualifier-seq_opt
    ref-qualifier_opt exception-specification_opt attribute-specifier-seq_opt
```

and the type of the contained *declarator-id* in the declaration T D1 is "derived-declarator-type-list T", the type of the *declarator-id* in D is "derived-declarator-type-list **`noexcept`$_{opt}$** function of (parameter-declaration-clause) *cv-qualifier-seq*$_{opt}$ *ref-qualifier*$_{opt}$ returning T"**, where the optional `noexcept` is present if and only if the *exception-specification* is non-throwing**. The optional *attribute-specifier-seq* appertains to the function type.

In a declaration T D where D has the form

```
D1 ( parameter-declaration-clause ) cv-qualifier-seq_opt
    ref-qualifier_opt exception-specification_opt attribute-specifier-seq_opt trailing-return-type
```

and the type of the contained *declarator-id* in the declaration T D1 is "derived-declarator-type-list T", T shall be the single *type-specifier* auto. The type of the *declarator-id* in D is "derived-declarator-type-list **`noexcept`$_{opt}$** function of (parameter-declaration-clause) *cv-qualifier-seq*$_{opt}$ *ref-qualifier*$_{opt}$ returning *trailing-return-type*"**, where the optional `noexcept` is present if and only if the *exception-specification* is non-throwing**. The optional *attribute-specifier-seq* appertains to the function type.

Change in 8.3.5 [dcl.fct] paragraph 5:

... After determining the type of each parameter, any parameter of type "array of T" or ~~"function returning T"~~ **of function type T** is adjusted to be "pointer to T" ~~or "pointer to function returning T," respectively~~. ...

Change in 8.3.5 [dcl.fct] paragraph 6:

... The return type, the parameter-type-list, the *ref-qualifier*, ~~and~~ the *cv-qualifier-seq*, **and whether the function has a non-throwing *exception-specification*,** but not the default arguments (8.3.6 [dcl.fct.default]) or the *exception specification* (15.4 [except.spec]), are part of the function type. ...

Change in section 8.4.1 [dcl.fct.def.general] paragraph 2:

The *declarator* in a *function-definition* shall have the form

```
D1 ~~( parameter-declaration-clause ) cv-qualifier-seq_opt~~
~~ref-qualifier_opt exception-specification_opt attribute-specifier-seq_opt~~
parameters-and-qualifiers trailing-return-type_opt
```

[Drafting note: This is intended to reduce the grammar redundancies around function declarators.]

Change in 8.5.3 [dcl.init.ref] paragraph 1:

A variable **whose** declared **type is** ~~to be a T& or T&&, that is,~~ "reference to type T" (8.3.2)~~;~~ shall be initialized ~~by an object, or function, of type T or by an object that can be converted into a T~~. [Example:

```
int g(int) noexcept;
...
```

]

Change in 8.5.3 [dcl.init.ref] paragraph 4:

Given types "cv1 T1" and "cv2 T2", "cv1 T1" is *reference-related* to "cv2 T2" if T1 is the same type as T2, or T1 is a base class of T2. "cv1 T1" is *reference-compatible* with "cv2 T2" if

- T1 is reference-related to T2 **or**
- **T2 is "`noexcept` function" and T1 is "function", where the function types are otherwise the same,**

and cv1 is the same cv-qualification as, or greater cv-qualification than, cv2. In all cases where the reference-related or reference-compatible relationship of two types is used to establish the validity of a reference binding, and T1 is a base class of T2, a program that necessitates such a binding is ill-formed if T1 is an inaccessible (Clause 11) or ambiguous (10.2) base class of T2.

Clause 13 [over] Overloading

Change in 13.1 [over.load] paragraph 2:

Certain function declarations cannot be overloaded:

- Function declarations that differ only in the return type**, the exception specification (15.4 [except.spec]), or both** cannot be overloaded.
- ...

No change in 13.3.3.1.1 [over.ics.scs] paragraph 1:

> ... [Note: These categories are orthogonal with respect to value category, cv-qualification, and data representation: the Lvalue Transformations do not change the cv-qualification or data representation of the type; the Qualification Adjustments do not change the value category or data representation of the type; and the Promotions and Conversions do not change the value category or cv-qualification of the type. -- end note]

In 13.3.3.1.1 [over.ics.scs], add an entry to table 11 "Conversions":

- **Conversion: Function pointer conversion**
- **Category: Qualification adjustment**
- **Rank: Exact Match**
- **Subclause: 4.12 [conv.fctptr]**

Change in 13.3.3.1.1 [over.ics.ics] paragraph 2:

> [Note: ... ~~At most one conversion from each category is allowed in a single standard conversion sequence.~~ ...]

Change in 13.4 [over.over] paragraph 1:

> ... ~~The function selected is the one whose type is identical to the function type of the target type required in the context.~~ **A function with type F is selected for the function type FT of the target type required in the context if F (after possibly applying the function pointer conversion (4.12 [conv.fctptr])) is identical to FT.** [Note: ...]

Change in 13.4 paragraph 3:

> ... Non-member functions and static member functions match targets of ~~pointer to function~~ **function pointer** type or reference to function type. ...

Change in 13.4 [over.over] paragraph 7:

> [Note: ~~There are no standard conversions (Clause 4) of one pointer-to-function type into another. In particular, even~~ **Even** if B is a public base of D, we have
>
> ```
> D* f();
> B* (*p1)() = &f;     // error
> void g(D*);
> void (*p2)(B*) = &g; // error
> ```
>
>]

Clause 14 [temp] Templates

Change in 14.1 [temp.param] paragraph 8:

> A non-type template-parameter of type "array of T" or ~~"function returning T"~~ **of function type T** is adjusted to be of type "pointer to T" ~~or "pointer to function returning T", respectively~~. [Example: ...]

Change in 14.8.2.1 [temp.deduct.call] paragraph 4:

> ... However, there are three cases that allow a difference:
>
> - ...
> - The transformed A can be another pointer or pointer to member type that can be converted to the deduced A via a **function pointer conversion (4.12 [conv.fctptr]) and/or** a qualification conversion (4.4 [conv.qual]).
> - ...

Change in 14.8.2.1 [temp.deduct.call] paragraph 6:

> When P is a function type, ~~pointer to function~~ **function pointer** type, or pointer to member function type: ...

Change in 14.8.2.3 temp.deduct.conv paragraph 5:

> In general, the deduction process attempts to find template argument values that will make the deduced A identical to A. However, there are ~~two~~ **four** cases that allow a difference:

- If the original A is a reference type, A can be more cv-qualified than the deduced A (i.e., the type referred to by the reference).
- **If the original A is a function pointer type, A can be "pointer to function" even if the deduced A is "pointer to `noexcept` function".**
- **If the original A is a pointer to member function type, A can be "pointer to member of type function" even if the deduced A is "pointer to member of type `noexcept` function".**
- ...

Clause 15 [except] Exception handling

Change in 15.3 except.handle paragraph 3:

> A *handler* is a match for an exception object of type E if
>
> - ...
> - the handler is of type cv T or const T& where T is a pointer type and E is a pointer type that can be converted to T by ~~either or both of~~ **one or more of**
>   - a standard pointer conversion (4.10 [conv.ptr]) not involving conversions to pointers to private or protected or ambiguous classes
>   - **a function pointer conversion (4.12 [conv.fctptr])**
>   - a qualification conversion **(4.4 [conv.qual])**
> - ...

Change in 15.4 [except.spec] paragraph 1:

> A function ~~declaration~~ **declarator** lists exceptions that its function might directly or indirectly throw by using an *exception-specification* as a suffix ~~of its declarator~~.

Change in 15.4 [except.spec] paragraph 2:

> ~~An *exception-specification* shall appear only on a function declarator for a function type, pointer to function type, reference to function type, or pointer to member function type that is the top-level type of a declaration or definition, or on such a type appearing as a parameter or return type in a function declarator. An exception-specification shall not appear in a typedef declaration or alias-declaration. [Example: ...]~~ ...

Change in 15.4 [except.spec] paragraph 5 and delete paragraph 6:

> [Example: ...] ~~A similar restriction applies to assignment to and initialization of pointers to functions, pointers to member functions, and references to functions: the target entity shall allow at least the exceptions allowed by the source value in the assignment or initialization. [Example: ...]~~
>
> ~~In such an assignment or initialization, *exception-specifications* on return types and parameter types shall be compatible. In other assignments or initializations, *exception-specifications* shall be compatible.~~

Change in 15.4 [except.spec] paragraph 15 bullet 1:

- ...
  - ...
  - **Otherwise, if the *postfix-expression* has type "noexcept function" or "pointer to noexcept function", S is the empty set.**
  - Otherwise, S contains the pseudo-type "any".
- ...

Add a new paragraph after 15.4 [except.spec] paragraph 15:

> **A function with an implied non-throwing exception specification, where the function's type is declared to be T, is instead considered to be of type "`noexcept` T".**

Change in 20.9.2 [func.require] paragraph 3 bullet 1:

- if T is a ~~pointer to function~~ **function pointer type, result_type shall be a synonym for the return type of T;**
- ...

Change in 20.9.4 [refwrap] paragraph 4 bullet 1:

- a function type or a ~~pointer to function~~ **function pointer** type taking one argument of type T1
- ...

Change in 20.9.4 [refwrap] paragraph 5 bullet 1:

- a function type or a ~~pointer to function~~ **function pointer** type taking two arguments of types T1 and T2
- ...

Add a new section after C.4.1 [diff.cpp14.lex]:

C.4.2 Clause 8: Declarators [diff.cpp14.decl]
8.3.5 [dcl.fct]

Change: Make exception specifications be part of the type system.
Rationale: Improve type-safety.
Effect on original feature: Valid C++2014 code may fail to compile or change meaning in this International Standard:

```
void g1() noexcept;
void g2();
template<class T> int f(T *, T *);
int x = f(g1, g2);    // ill-formed; previously well-formed
```